University of Pittsburgh

CS 2410 Introduction

Technology Advances

Wonsun Ahn
Department of Computer Science
School of Computing and Information

Technology Advances

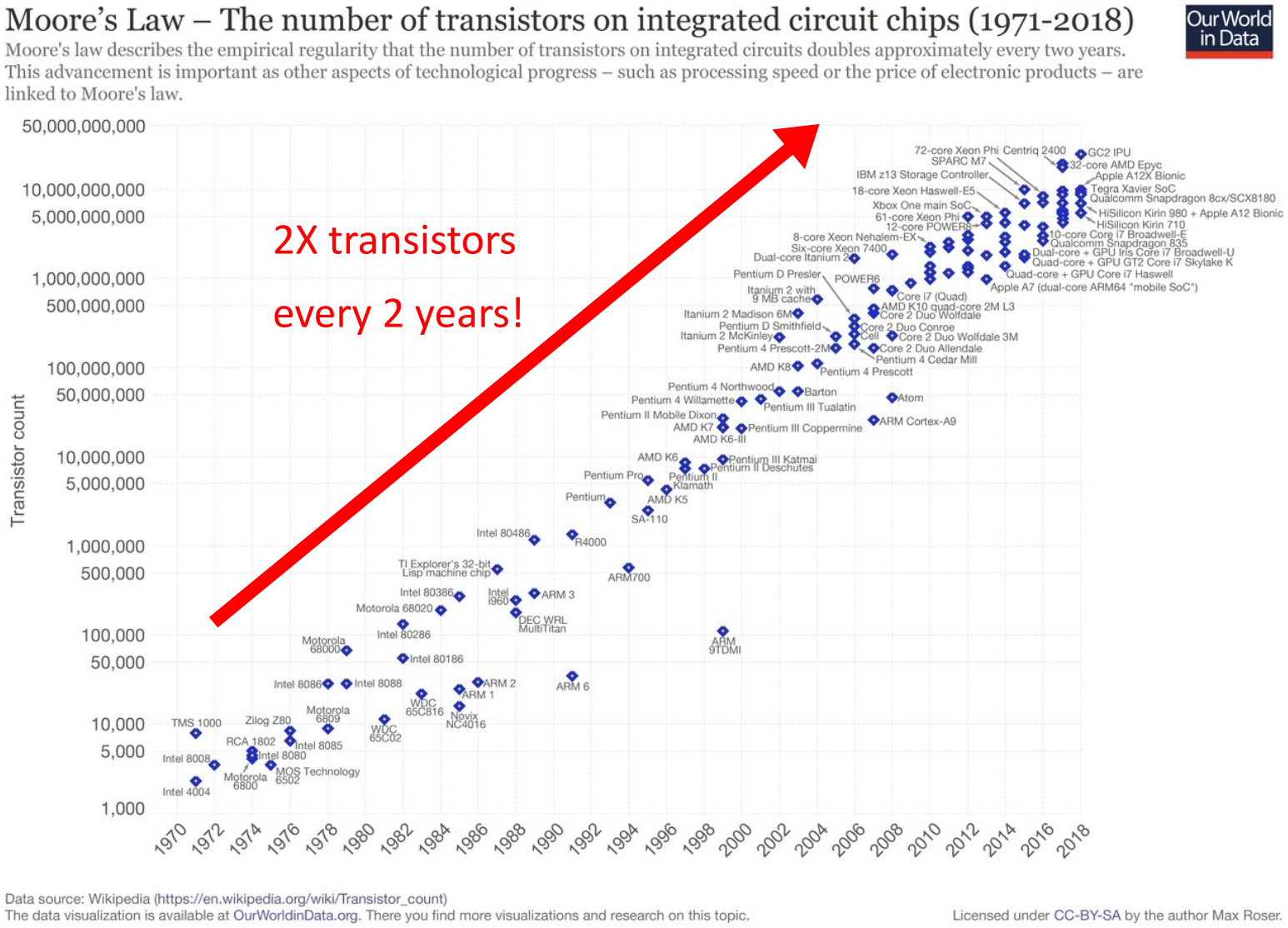
Advances in Technology

- Technology has been advancing at lightning speed
- Architecture and IT as a whole were beneficiaries

- Technology advance is summarized by *Moore's Law*
  - You probably heard of it at some point. Something about …
  - "X doubles every 18-24 months at constant cost"

- Is X:
  - CPU performance?
  - CPU clock frequency?

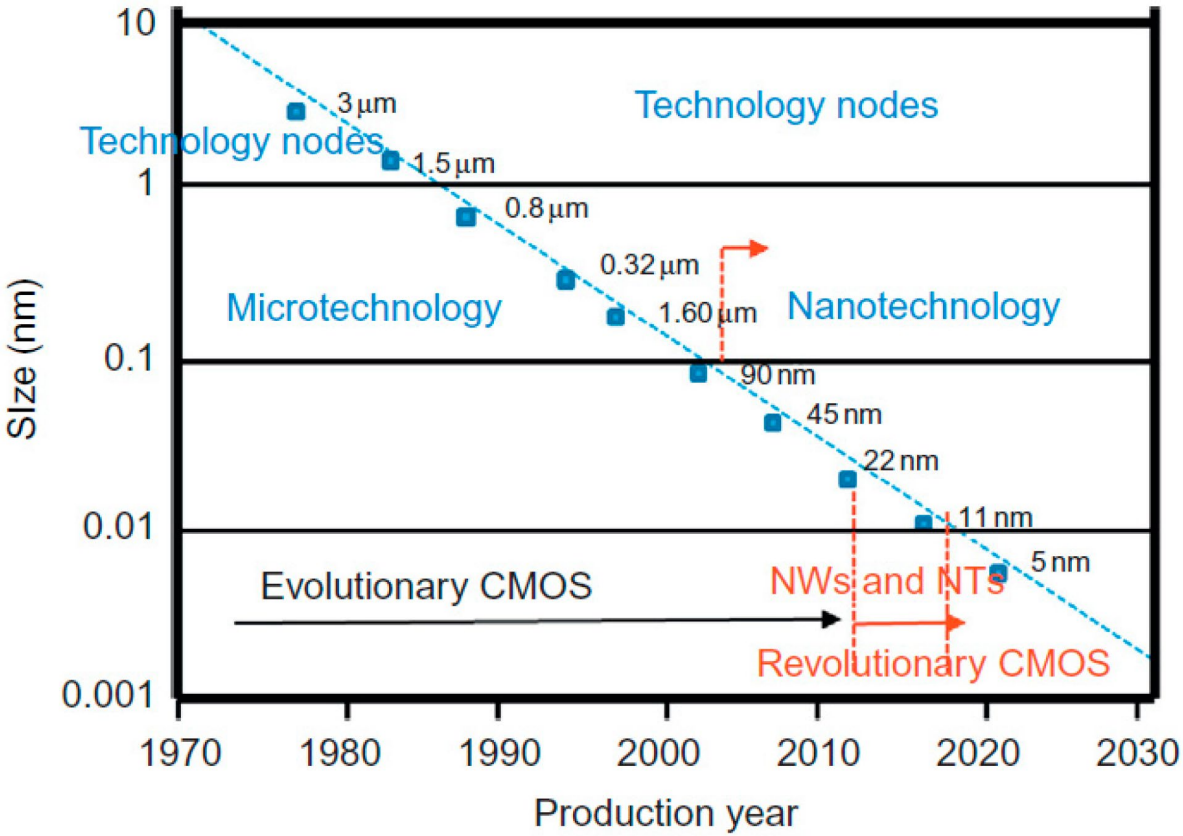
Moore's Law



Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.

Miniaturization of Transistors



Data source: Radamson, H.H.; He, X.; Zhang, Q.; Liu, J.; Cui, H.; Xiang, J.; Kong, Z.; Xiong, W.; Li, J.; Gao, J.; Yang, H.; Gu, S.; Zhao, X.; Du, Y.; Yu, J.; Wang, G. Miniaturization of CMOS. *Micromachines* **2019**, *10*, 293.

- Moore's Law has been driven by transistor miniaturization
  - CPU chip area hasn't changed much

Future of Moore's Law

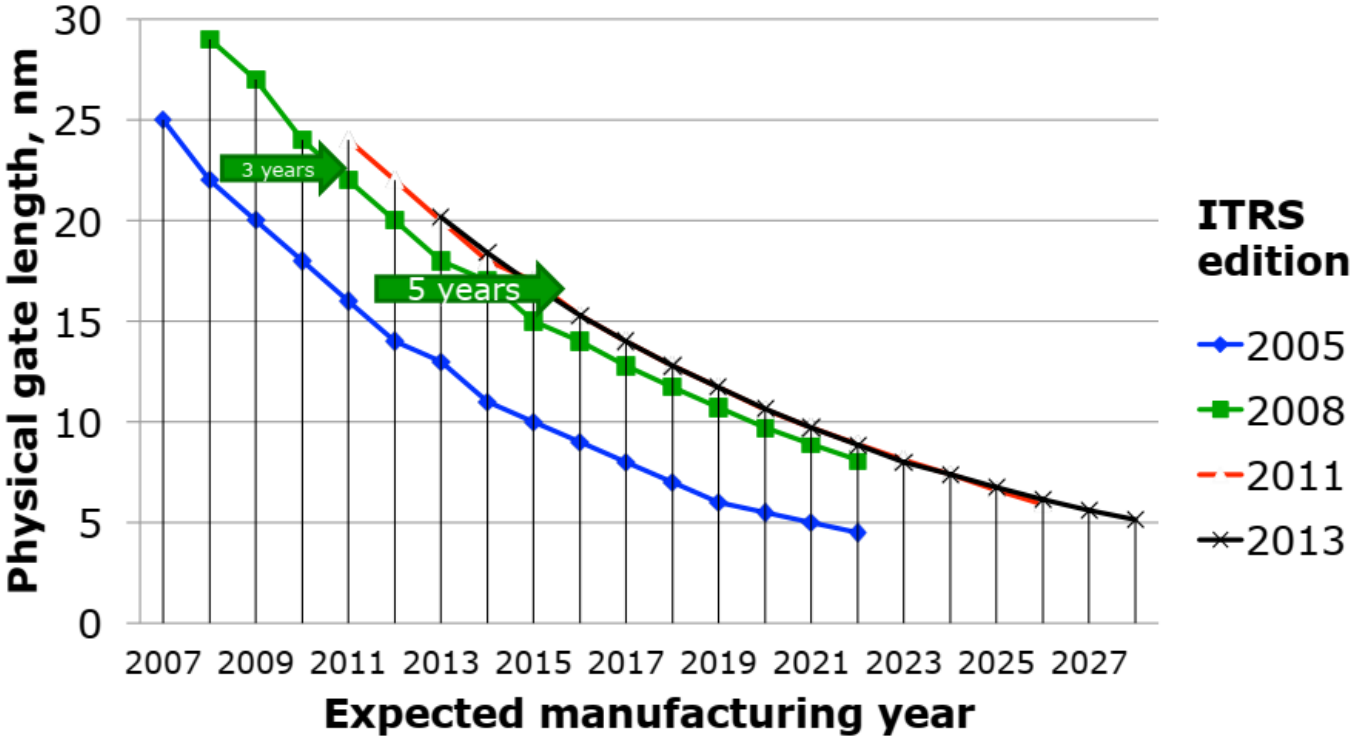
- The semiconductor industry has produced roadmaps
  - Semiconductor Industry Association (SIA): 1977~1997
  - International Technology Roadmap for Semiconductors (ITRS): 1998~2016
  - International Roadmap for Devices and Systems (IRDS): 2017~Present
- IRDS Lithography Projection (2024)

Year of Production	2018	2020	2022	2025	2027	2029	2031
Technology Node (nm)	7	5	3	2	1.4	1.0	0.7

  - Moore's Law will continue into foreseeable future
  - IRDS does not project significant increase in CPU chip size
  - Increases in transistors will come from *transistor density*

IRDS isn't Perfect

- ITRS (predecessor of IRDS) has made corrections before



  - After all, you are trying to predict the future
  - But architects rely on the roadmap to design future processors

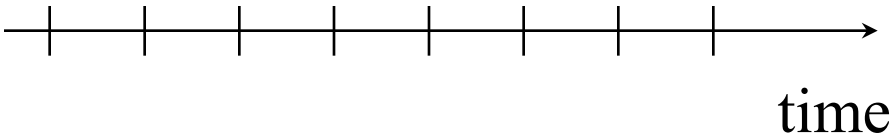
Moore's Law and Performance

- Did Moore's Law result in higher performance CPUs?
- How do you measure performance?
  - When you buy a CPU, what number(s) do you look at?

1. CPU parameters (clock frequency, width, cache size)
   - These numbers do not always translate to performance
2. Try running your favorite apps on it and measure time.
   - Most accurate, but manufacturers can't publish these numbers
3. Try running a suite of benchmark apps (e.g. SPECCPU)
   - Less accurate, but it is an objective publishable number
   - ☞ How performance is measured in research and industry

Components of Execution Time

- Processor activity happens on clock "ticks" or cycles



  - On each tick, bits flow through logic gates and are latched

- Execution time = $\frac{\text{seconds}}{\text{program}}$

$$\frac{\text{seconds}}{\text{program}} = \frac{\text{cycles}}{\text{program}} \times \frac{\text{seconds}}{\text{cycle}}$$

$$= \frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

How to improve Execution Time

$$\frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

- Reduce $\frac{\text{seconds}}{\text{cycle}}$: a.k.a. CCT (Clock Cycle Time)
  - Clock frequency (GHz) = $\frac{\text{cycles}}{\text{second}}$ = reverse of $\frac{\text{seconds}}{\text{cycle}}$
  - In other words, increase clock frequency
- Reduce $\frac{\text{cycles}}{\text{instruction}}$: a.k.a. CPI (Cycles Per Instruction)
  - IPC (Instructions Per Cycle) = $\frac{\text{instructions}}{\text{cycle}}$ = reverse of $\frac{\text{cycles}}{\text{instruction}}$
  - In other words, increase IPC
- Reduce $\frac{\text{instructions}}{\text{program}}$:
  - Less instructions leads to shorter execution time
  - ISAs that do a lot of work with one instruction shortens time

Moore's Law impacts two layers

- Does Moore's Law result in higher performance CPUs?
- Law impacts both architecture and physical layers

Instruction Set Architecture	Computer Architecture
Microarchitecture	
Transistor Implementation	Physical Layer

- Microarchitecture: many more transistors to implement ISA
- Transistor Implementation: smaller, more efficient transistors

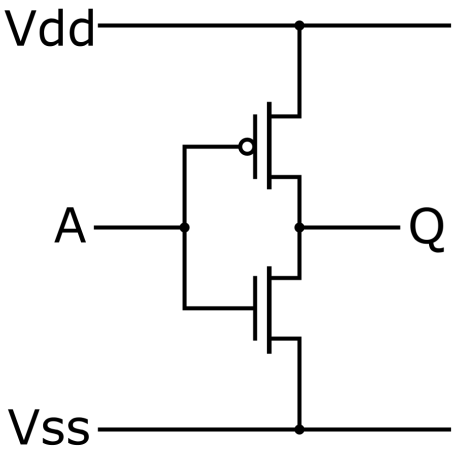
How Moore's Law impacts performance

- Clock Cycle Time = transistor delay × number of transistors between clock ticks
  - Transistor delay → transistor implementation
  - Number of transistors between clock ticks → microarchitecture
- Cycler Per Instruction (CPI) → microarchitecture
- Instructions/Program → instruction set architecture

- More transistors enables better microarchitecture
  - We will study how throughout the semester ☺
- Smaller transistors have reduced transistor delay
  - ☞ This is what we will try to understand this chapter

Gates using CMOS transistors

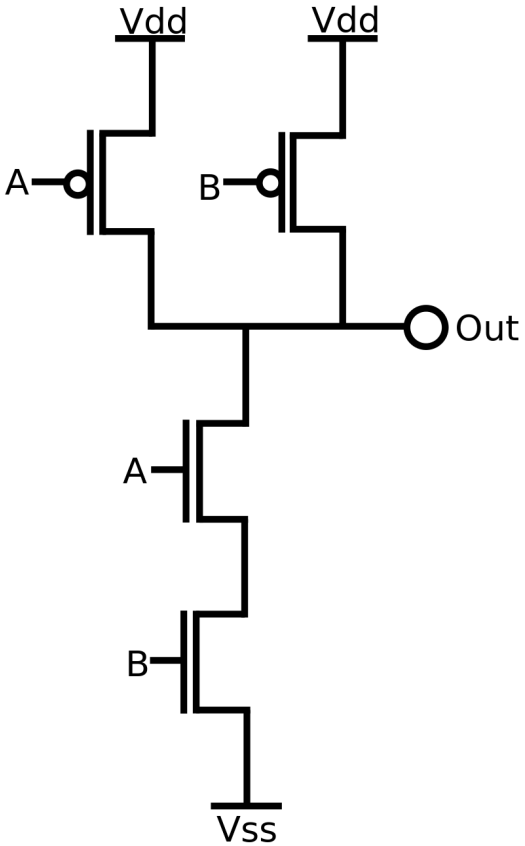
- NOT gate



Vdd = Supply voltage

Vss = Ground voltage

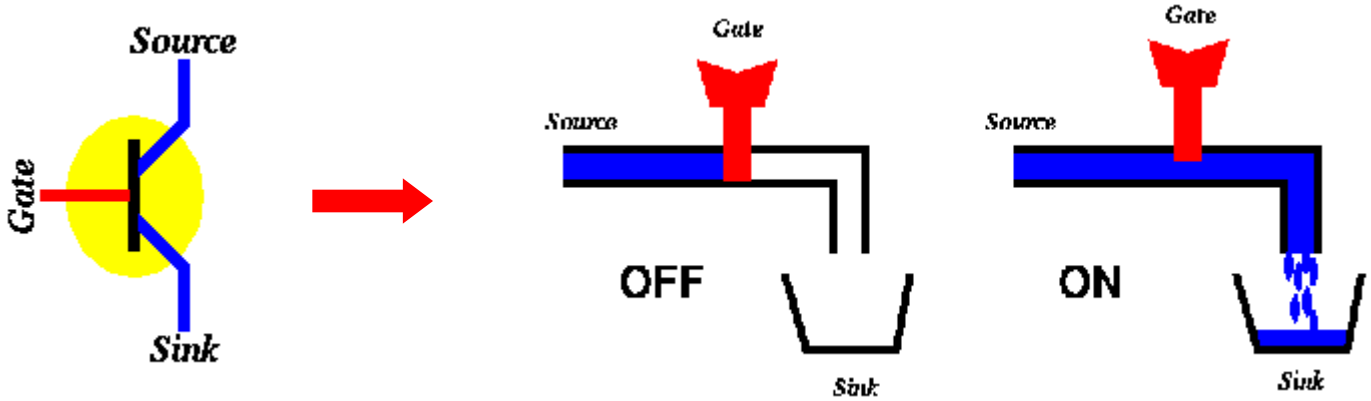
- NAND gate



- Gate delay = how fast input propagates to output

Transistor Delay

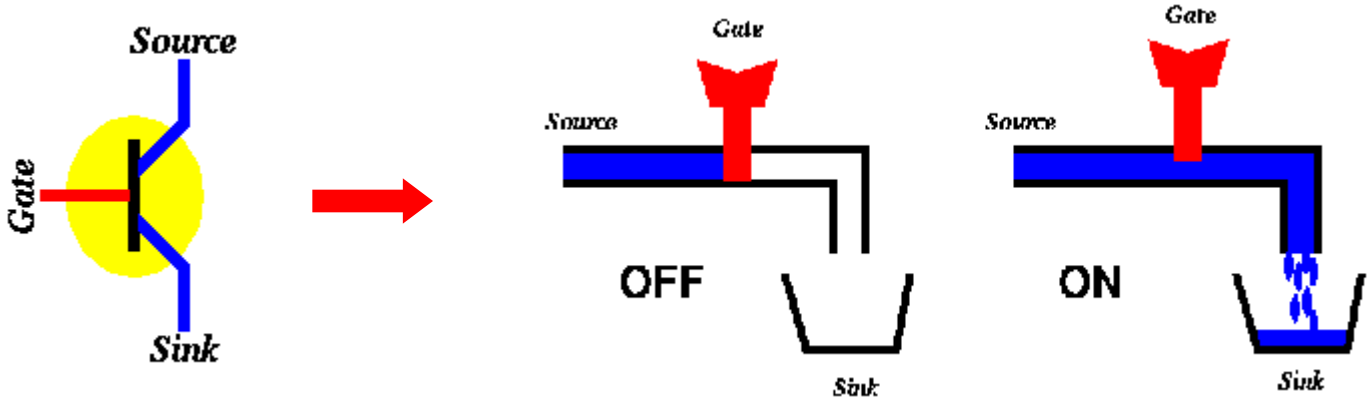
- Transistors are like faucets:



- Gate is the "input" value
- Sink is the "output" value
- Delay → time for water in bucket to reach threshold
  - At this point, the output is turned "on"
  - Output serves as input (gate) to the next transistor

Transistor Delay

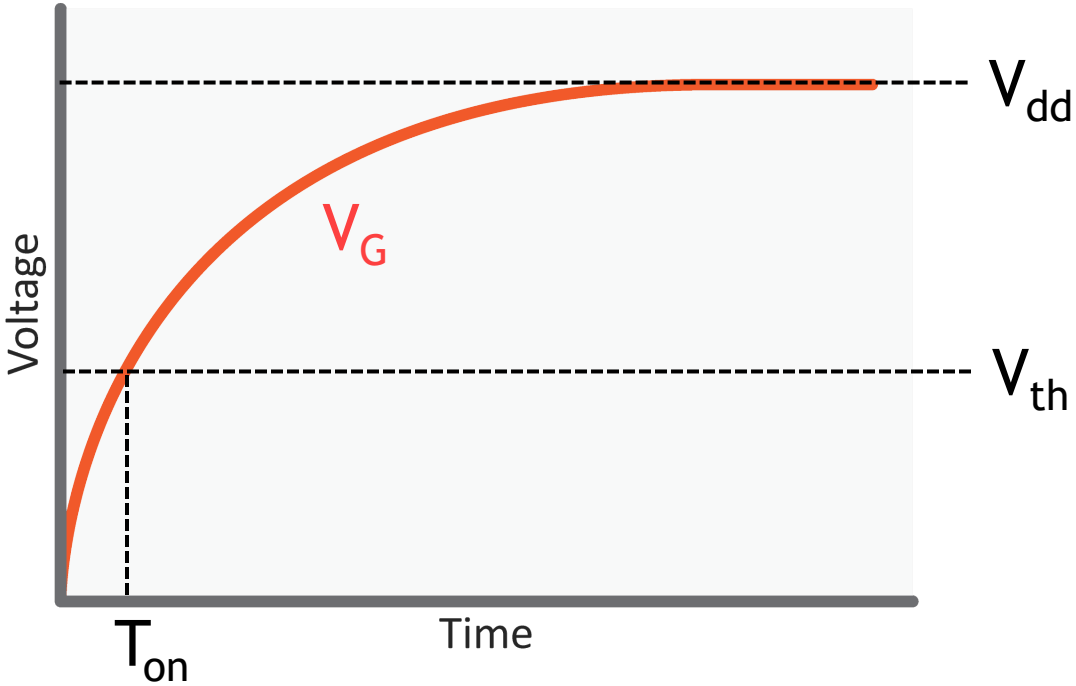
- Transistors are like faucets:



- To make a transistor go fast, do one of the following:
  - Increase water pressure (supply voltage)
  - Change faucet (transistor) design
- Transistor design can be changed in two ways:
  - Increase pipe thickness (reduce channel resistance R)
  - Reduce bucket size (reduce capacitance C)

Transistor delay (T_{on}) ∝ RC

- Charging Curve of V_G (Gate Voltage) when switching on

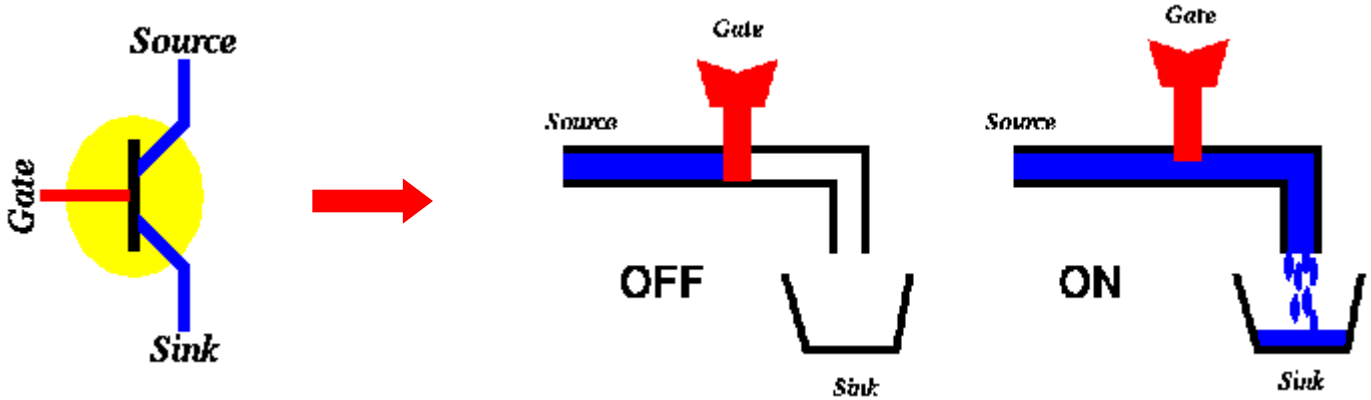


$$V_G = V_{dd}(1 - e^{\frac{t}{RC}})$$

- V_{th} determines the threshold V_G when switching happens
- T_{on} when V_G reaches V_{th} ∝ RC

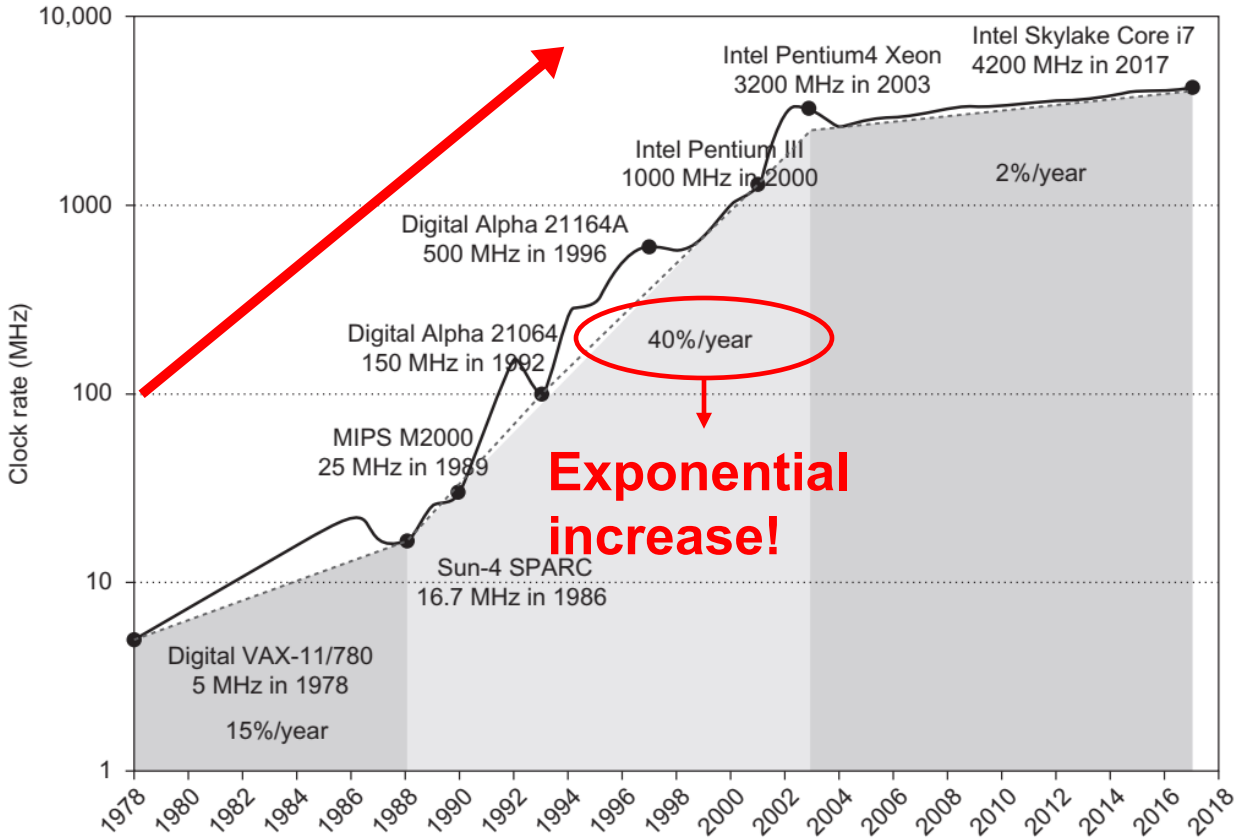
Smaller Transistors are Faster!

- Transistors are like faucets!



- For transistor dimension d (node size)
  - Channel resistance $R \propto$ Channel length $L \propto d$
  - Capacitance $C \propto d$
- Given the same *supply voltage*, smaller is faster!
- Did Moore's Law enjoy faster and faster frequencies?

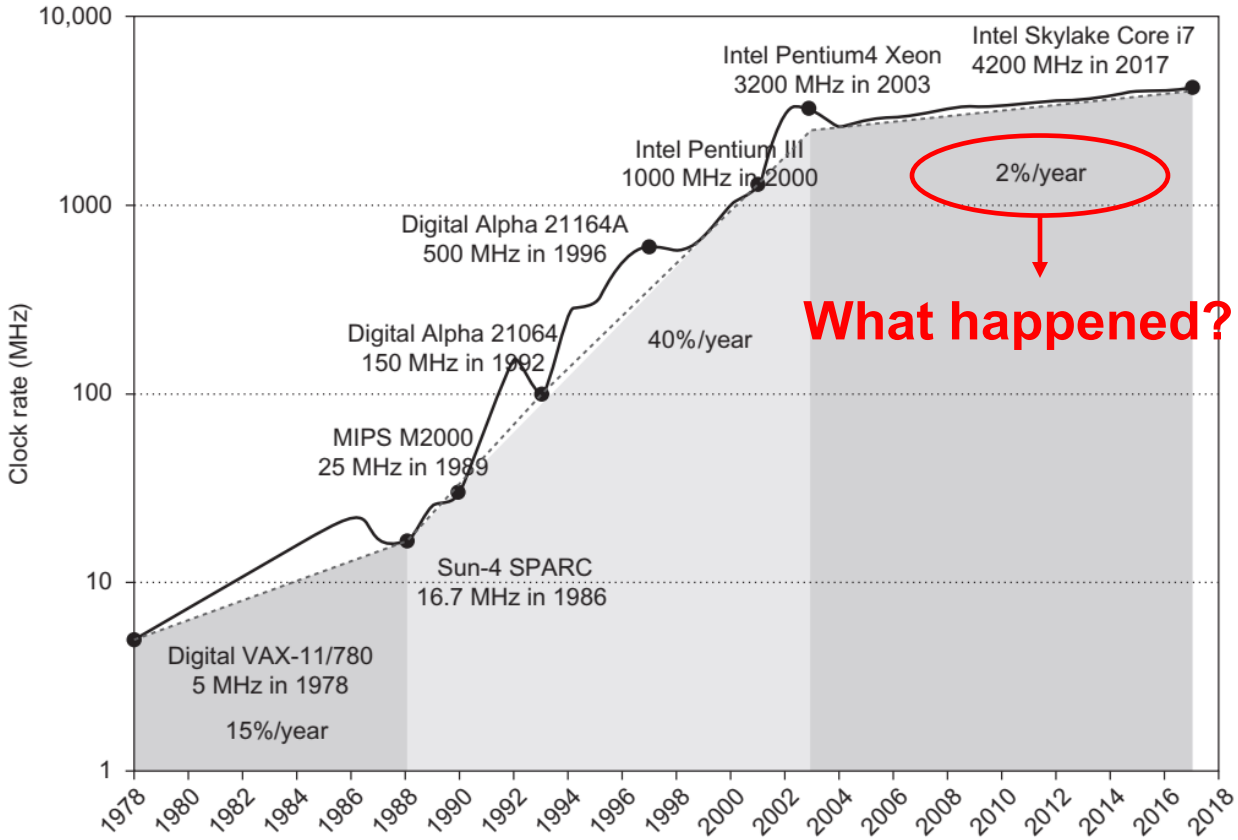
Yes, for a while …



Source: Computer Architecture, A Quantitative Approach (6th ed.) by John Hennessy and David Patterson, 2017

- Due in large part to improvement in transistor speed
  - CPU design (pipelining) contributed but we'll discuss later

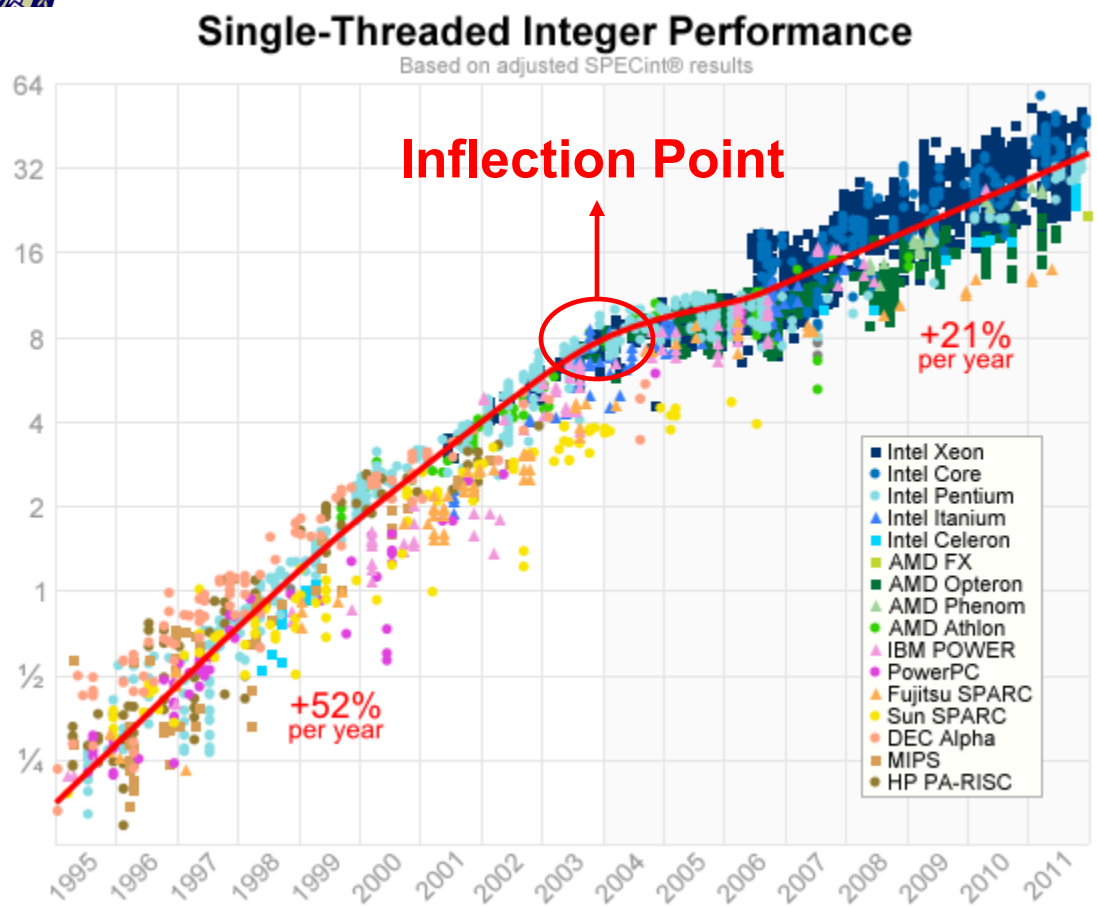
But not so much lately



Source: Computer Architecture, A Quantitative Approach (6th ed.) by John Hennessy and David Patterson, 2017

- Suddenly around 2003, frequency scaling stops. Why?
  - Improvements in transistor speed stopped.

Dent in CPU Performance



Source: https://preshing.com/20120208/a-look-back-at-single-threaded-cpu-performance/

- This caused a big dent in CPU performance circa 2003
- Improvements henceforth came from microarchitecture (for the most part)

Why did frequency scaling stop? TDP.

- *TDP* (*Thermal Design Power*):
  - Maximum power (heat) that CPU is designed to generate
  - Capped by the amount of heat cooling system can handle
  - Cooling system hasn't improved much over generations

- CPU Power = A * N * CFV^2 must be < TDP
  - A = Activity factor (% of transistors with activity)
  - N = Number of transistors
  - C = Capacitance
  - F = Frequency
  - V = Supply Voltage

- What happens to each factor with Moore's Law?

Phase 1: Fixed Voltage (until ~1993)

- Change in CPU Power $\propto$ A * N * CFV^2, with fixed V:
  - A = Activity factor
  - N = Number of transistors $\propto 1/d^2$ ⇧ ⇧
  - C = Capacitance (size of bucket) $\propto$ d ⇩
  - F $\propto 1/T_{on} \propto 1/RC \propto 1/d^2$ ⇧ ⇧ (max allowed by transistor speedup)
  - V = Supply Voltage (water pressure kept constant)

  → CPU Power $\propto 1/d^3$ ⇧ ⇧ ⇧ !

- Recipe used until 1990's until CPUs became too hot
  - With CPUs in melt down, could no longer clock F at full speed

Q) So how did CPU frequency keep increasing up to 2003?

Phase 2: Dennard Scaling (until ~2003)

- Dennard Scaling: a new recipe for scaling up frequency, while reducing supply voltage to keep power constant

- By reducing V proportional to d,
  then changes in CPU Power $\propto$ A * N * CFV^2 are:
  - A = Activity factor
  - N = Number of transistors $\propto 1/d^2$ ⇧ ⇧
  - C = Capacitance $\propto$ d ⇩
  - F $\propto$ 1/d ⇧ (can now only scale by 1/d to keep power constant)
  - V = Supply Voltage $\propto$ d ⇩ $\rightarrow$ $V^2 \propto d^2$ ⇩ ⇩

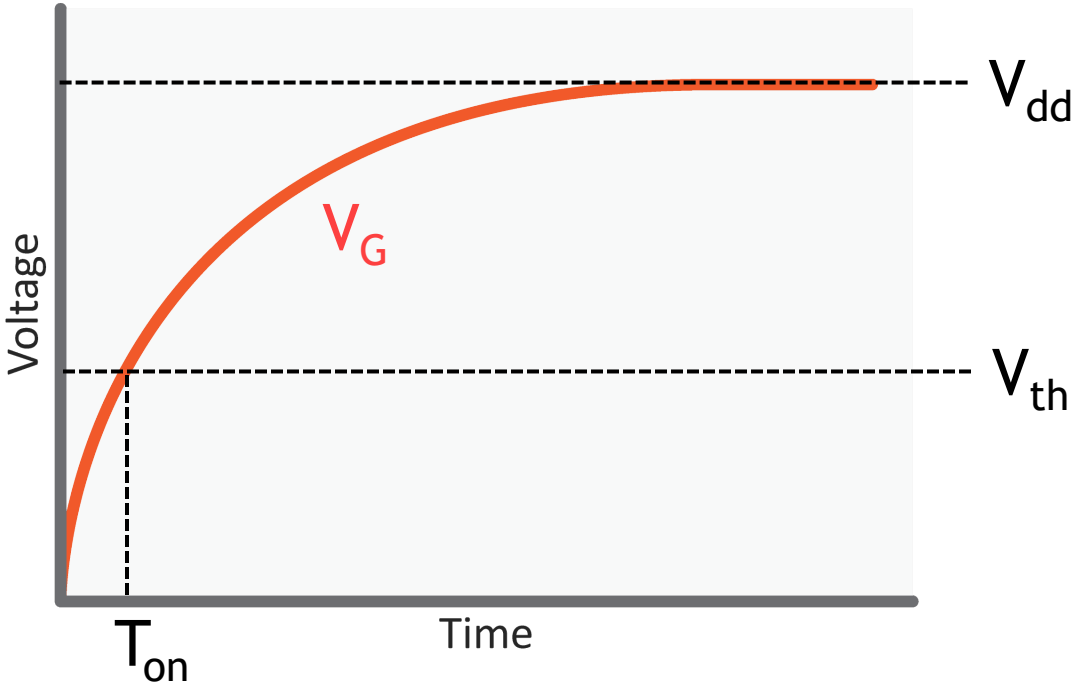
$\rightarrow$ Effects cancel each other out $\rightarrow$ power is kept constant

Dennard Scaling and V_{th}

- So, it's that easy? Just reduce V until you meet TDP?
- No, it's not that simple ☹.
- Reducing V_{dd} (supply voltage) affects CPU operation
  - As V_{dd} is reduced, transistor becomes slower (lower pressure)
  - Eventually, CPU stops working altogether
- Transistors need redesigning to work at lower voltage
  - V_{th} is the threshold voltage when switching happens
  - V_{th} needs to be reduced along with V_{dd} to maintain speed

V_{dd} and V_{th} determine transistor speed

- RC Charging Curve of V_G (Gate Voltage)

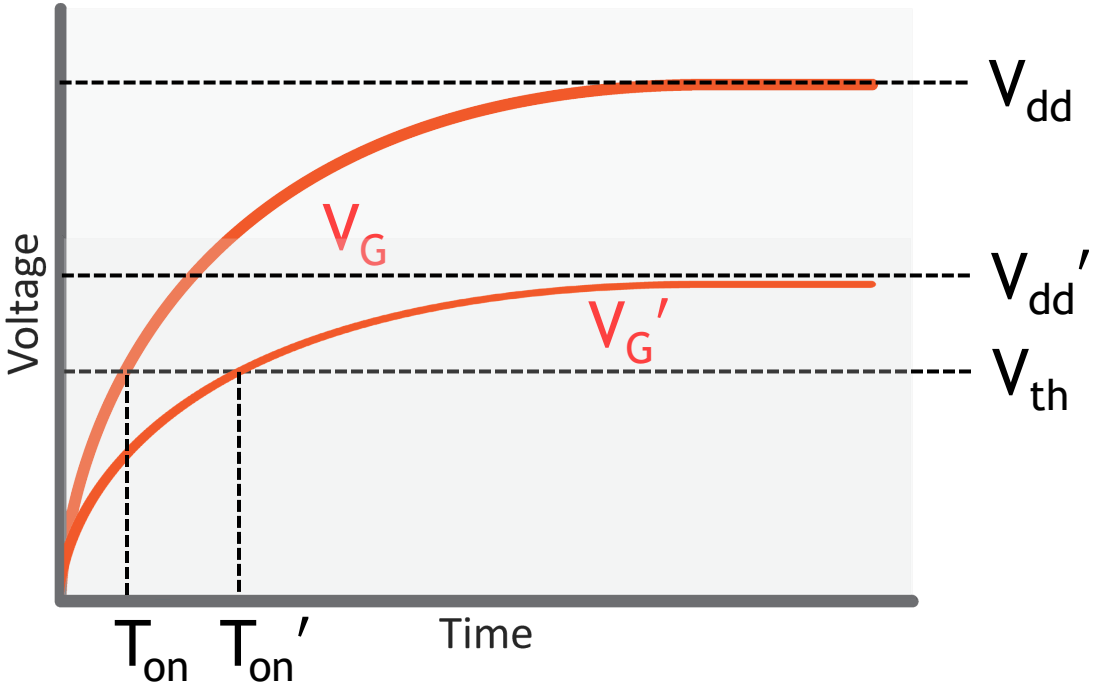


$$V_G = V_{dd}(1 - e^{\frac{t}{RC}})$$

- V_{dd} determines how quickly the transistor charges up
- V_{th} determines the threshold V_G where switching happens

Lower V_{dd}⇩ means lower speed⇩

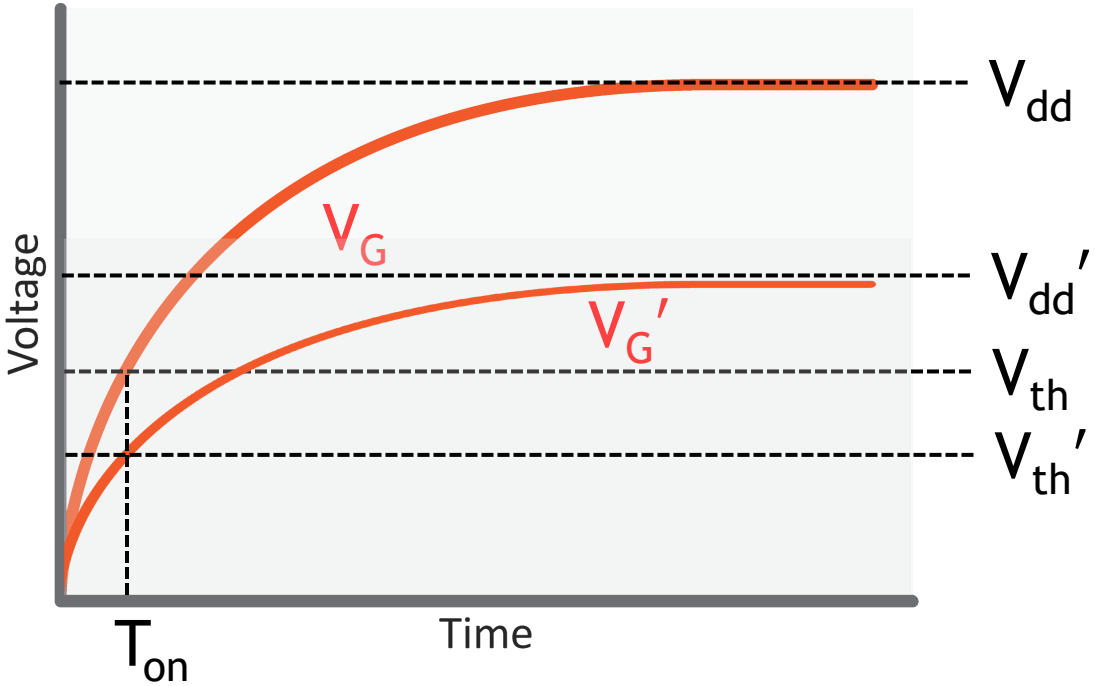
- RC Charging Curve of V_G (Gate Voltage)



- V_{dd} is high → V_G reaches V_{th} quickly at T_{on} (high pressure)
- V_{dd}' is low → V_G reaches V_{th} slowly at T_{on}'(low pressure)

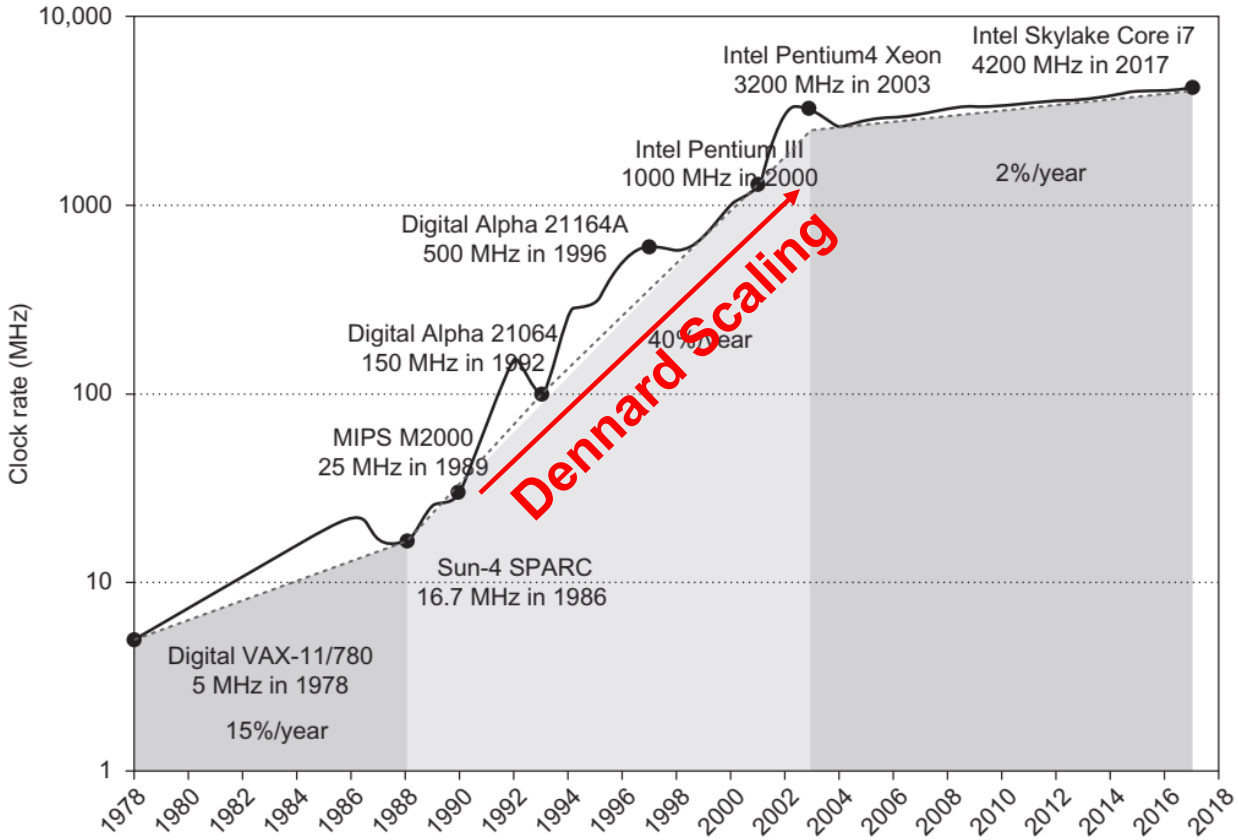
Speed is maintained only with lower V_{th}⇩

- RC Charging Curve of V_G



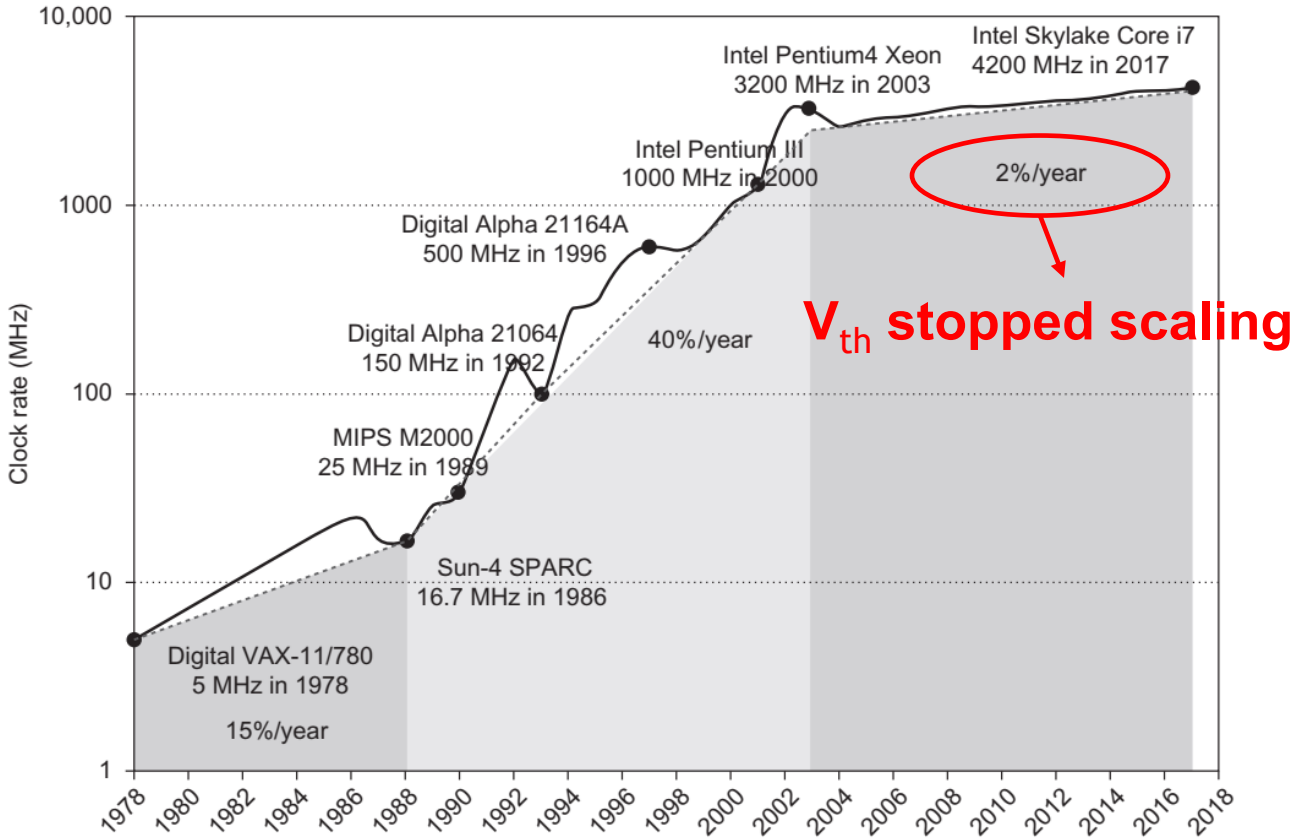
- Speed (T_{on}) is maintained while reducing V_{dd} to V_{dd}', but only if V_{th} is also reduced to V_{th}'

Dennard Scaling paid dividends for a while



- Aided by continuous reduction of V_{dd} and V_{th}

Then Dennard Scaling ended at 2003

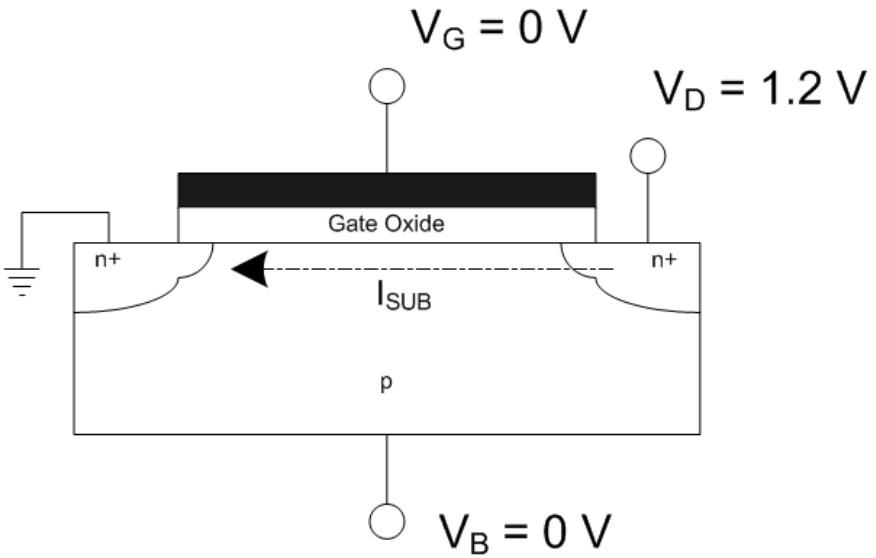


V_{th} stopped scaling

- Main reason was that V_{th} could no longer be reduced

Limits to Dropping V_{th}

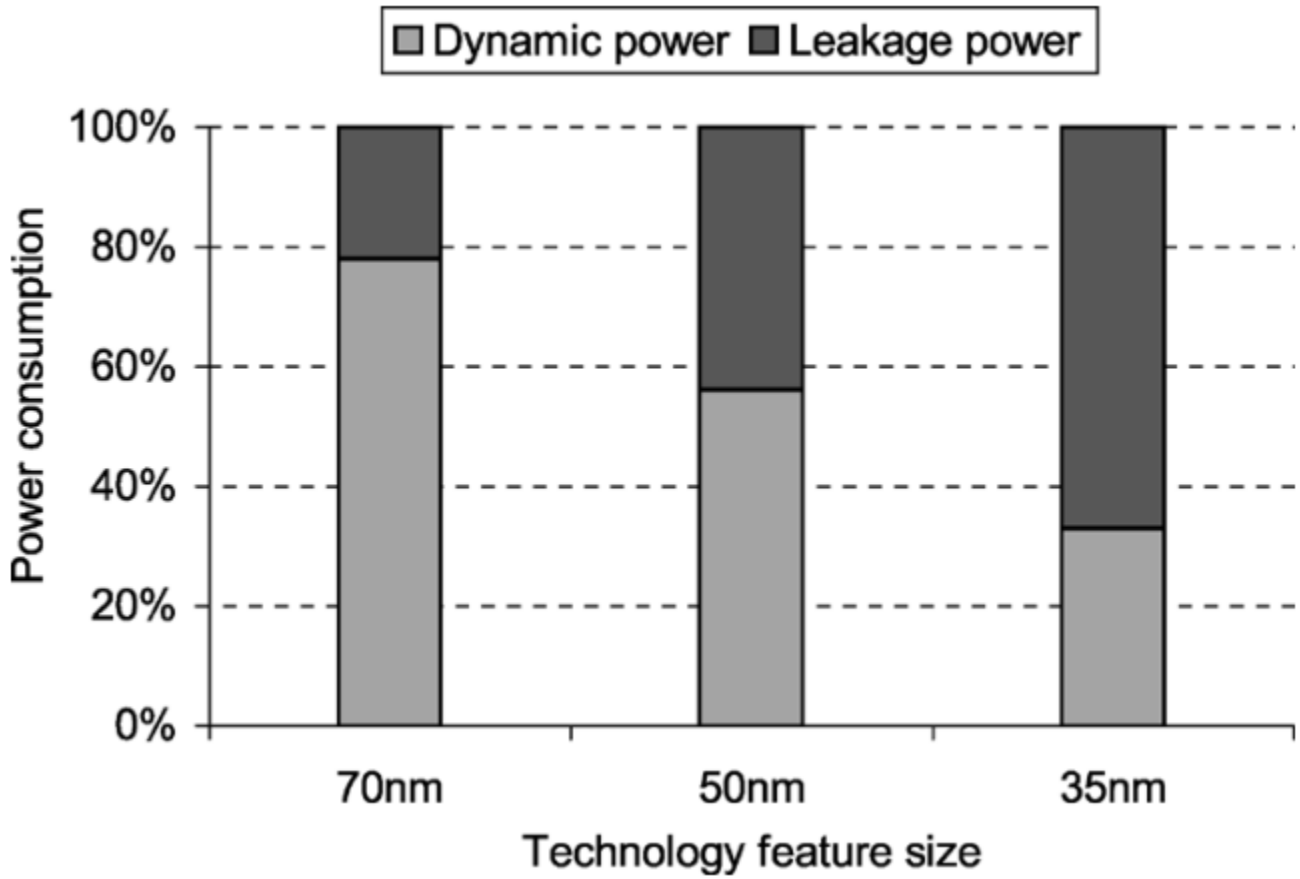
- Subthreshold leakage
  - Transistor leaks current even when gate is off ($V_G = 0$)



  - This leakage current translates to leakage power
  - Leakage worsens when V_{th} is dropped

Leakage Power across Generations

- Leakage power has increased across technology nodes



Source: L. Yan, Jiong Luo and N. K. Jha, "Joint dynamic voltage scaling and adaptive body biasing for heterogeneous distributed real-time embedded systems," in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 24, no. 7, pp. 1030-1041, July 2005

CPU Power must now include leakage

- Previous power calculation was incomplete
  - CPU power is the sum of both dynamic and leakage power
- $Power_{CPU} \propto Power_{dynamic} + Power_{leakage}$
  - $Power_{dynamic} \propto A * N * CFV_{dd}^2$
  - $Power_{leakage} \propto N * V_{dd} * e^{-Vth}$
  - Leakage power worsens *exponentially* when V_{th} is dropped

- Catch-22: to lower $Power_{dynamic}$, we need to lower V_{th}, but that leads to a larger increase of $Power_{leakage}$
  → V_{th} can no longer be lowered
  → V_{dd} can no longer be lowered either
  → Dennard Scaling must come to an end

Phase 3: End of Dennard Scaling (2003~)

- What happens to frequency without Dennard Scaling?
- $Power_{dynamic}$ ($\propto A * N * CFV^2$) + $Power_{leakage}$ ($\propto N * V * e^{-Vth}$)
  - A = Activity factor
  - N = Number of transistors ($\propto 1/d^2$) ⇧ ⇧
  - C = Capacitance ($\propto d$) ⇩
  - V = Supply Voltage ⇔ (Due to fixed V_{th})
  - F = Frequency ???
- To offset N, you actually have to *decrease* F
- Otherwise, if you want to maintain F, must decrease N
  - That is, you cannot power on all the transistors at any given point
  - **Dark silicon**: situation where chip is only partially powered

Free Ride is Over

- "Free" speed improvements from transistors is over
- Now it's up to architects to improve performance
  - Moore's Law is still alive (although slowing down)
  - Architects are flooded with extra transistors each generation
  - But it's hard to even keep them powered without reducing F!
- Now is a good time to discuss technology constraints
  - Since we already mentioned a big one: TDP